TUF+ DSA Checklist

Reorganized in a single continuous topic-wise order • with short optimal-solution hints • based on takeUforward A2Z DSA Sheet

★ = must-do / most important interview question • highlighted row = important • "Pattern" column = core technique/topic tag

Arrays (24 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
1	★ **Majority Element-I**	*Boyer-Moore voting algo, O(n)/O(1)*	**Boyer-Moore Voting**	■		■
2	Leaders in an Array	*Traverse right to left, track max so far*	**Suffix Max Scan**	■		■
3	★ **Rearrange array elements by sign**	*Two pointers for pos(0) & neg(1) index, fill result array*	**Two Pointers**	■		■
4	★ **Print the matrix in spiral manner**	*4 boundary pointers: top,bottom,left,right, shrink after each pass*	**Matrix Boundary Sim**	■		■
5	Pascal's Triangle I	*nCr formula for given row & col*	**Combinatorics**	■		■
6	Pascal's Triangle II	*Build one row using nCr(row,col), iterative multiply*	**Combinatorics**	■		■
7	Pascal's Triangle III	*Generate all rows, each built from nCr formula*	**Combinatorics**	■		■
8	★ **Rotate matrix by 90 degrees**	*Transpose matrix then reverse each row, in-place*	**Matrix In-place**	■		■
9	★ **Two Sum**	*HashMap store value->index, check complement while iterating*	**Hashing**	■		■
10	★ **3 Sum**	*Sort array, fix i, two-pointer on remaining, skip duplicates*	**Sort + Two Pointers**	■		■
11	★ **4 Sum**	*Sort array, fix i,j, two-pointer for remaining pair, skip dupes*	**Sort + Two Pointers**	■		■
12	★ **Sort an array of 0's 1's and 2's**	*Dutch National Flag: low/mid/high 3-pointer, single pass*	**Dutch National Flag**	■		■
13	★ **Kadane's Algorithm**	*Track running sum, reset to 0 if negative, track max*	**Kadane / DP**	■		■
14	★ **Next Permutation**	*Find break point from right, swap with next greater, reverse suffix*	**Array Simulation**	■		■
15	Majority Element-II	*Extended Boyer-Moore with two candidates, verify counts*	**Boyer-Moore Voting**	■		■

Arrays (24 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
16	★ **Find the repeating and missing number**	*XOR trick or sum/sum-of-squares equations, O(n)/O(1)*	**XOR / Math**	■		■
17	★ **Count Inversions**	*Merge sort, count cross-inversions during merge step*	**Merge Sort**	■		■
18	Reverse Pairs	*Merge sort, count pairs where a[i] > 2*a[j] before merging*	**Merge Sort**	■		■
19	★ **Maximum Product Subarray in an Array**	*Track running max & min product (handles negatives), reset on 0*	**Kadane Variant**	■		■
20	Merge two sorted arrays without extra space	*Gap method (Shell sort style) or swap-if-greater with two pointers*	**Gap Method**	■		■
21	★ **Longest Consecutive Sequence in an Array**	*HashSet, only start counting from numbers with no predecessor*	**Hashing**	■		■
22	★ **Longest subarray with sum K**	*Prefix sum + HashMap of first occurrence (handles negatives); two-pointer if all positive*	**Prefix Sum + Hashing**	■		■
23	★ **Count subarrays with given sum**	*Prefix sum frequency HashMap, count occurrences of (sum-K)*	**Prefix Sum + Hashing**	■		■
24	Count subarrays with given xor K	*Prefix XOR + HashMap, count occurrences of (xor^K)*	**Prefix XOR + Hashing**	■		■

Binary Search (28 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
25	Search X in sorted array	*Standard binary search, compare mid to target*	**Binary Search**	■		■
26	★ **Lower Bound**	*Binary search for first index with arr[i] >= x*	**Binary Search**	■		■
27	★ **Upper Bound**	*Binary search for first index with arr[i] > x*	**Binary Search**	■		■
28	Search insert position	*Same as lower bound binary search*	**Binary Search**	■		■
29	Floor and Ceil in Sorted Array	*Binary search: floor=last <=x, ceil=first >=x*	**Binary Search**	■		■
30	★ **First and last occurrence**	*Binary search twice: lower bound & upper bound-1*	**Binary Search**	■		■

Binary Search (28 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
31	★ **Search in rotated sorted array-I**	*Binary search, identify which half is sorted, narrow accordingly*	**Binary Search on Rotated Array**	■		■
32	Search in rotated sorted array-II	*Same as above; when arr[lo]==arr[mid]==arr[hi] shrink both ends*	**Binary Search on Rotated Array**	■		■
33	★ **Find minimum in Rotated Sorted Array**	*Binary search, move to side without the sorted-half minimum*	**Binary Search on Rotated Array**	■		■
34	Find out how many times the array is rotated	*Binary search for index of minimum element*	**Binary Search on Rotated Array**	■		■
35	Single element in sorted array	*Binary search on even/odd index pairing to locate break point*	**Binary Search on Index Parity**	■		■
36	Find square root of a number	*Binary search on answer space [1,n], find largest m with m*m<=n*	**Binary Search on Answer**	■		■
37	Find Nth root of a number	*Binary search on answer, check mid^n vs target*	**Binary Search on Answer**	■		■
38	Find the smallest divisor	*Binary search on divisor value, sum ceil(a[i]/d) <= limit*	**Binary Search on Answer**	■		■
39	★ **Koko eating bananas**	*Binary search on speed k, check hours needed <= H*	**Binary Search on Answer**	■		■
40	★ **Minimum days to make M bouquets**	*Binary search on days, greedily count adjacent-bloomed flowers*	**Binary Search on Answer**	■		■
41	★ **Aggressive Cows**	*Binary search on min distance, greedily place cows*	**Binary Search on Answer**	■		■
42	★ **Book Allocation Problem**	*Binary search on max pages, greedily count students needed*	**Binary Search on Answer**	■		■
43	★ **Find peak element**	*Binary search using slope: move to higher neighbor side*	**Binary Search on Slope**	■		■
44	★ **Median of 2 sorted arrays**	*Binary search on smaller array partition, O(log(min(n,m)))*	**Binary Search on Partition**	■		■
45	Kth element of 2 sorted arrays	*Binary search partition variant of median-of-two-sorted-arrays*	**Binary Search on Partition**	■		■

Binary Search (28 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
46	Minimize Max Distance to Gas Station	*Binary search on distance (real-valued) + greedy feasibility check*	**Binary Search on Answer**	■		■
47	★ **Split array - largest sum**	*Binary search on answer, greedy count of subarrays needed*	**Binary Search on Answer**	■		■
48	Find row with maximum 1's	*Binary search each row for first 1, or staircase search from top-right*	**Binary Search / Staircase**	■		■
49	★ **Search in a 2D matrix**	*Treat as 1D sorted array, binary search with index mapping*	**Binary Search 2D**	■		■
50	★ **Search in 2D matrix - II**	*Start top-right corner, move left if bigger, down if smaller*	**Staircase Search**	■		■
51	Find Peak Element - II	*Binary search on columns, find max in column, move toward higher neighbor*	**Binary Search 2D**	■		■
52	Matrix Median	*Binary search on value range, count elements <= mid per row (upper bound)*	**Binary Search on Answer**	■		■

Recursion & Backtracking (17 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
53	★ **Pow(x,n)**	*Fast exponentiation: divide n by 2 recursively, square base*	**Fast Exponentiation**	■		■
54	★ **Generate Parentheses**	*Backtrack tracking open/close counts, add ')' only if valid*	**Backtracking**	■		■
55	Power Set	*Recursion/bitmasking: include/exclude each element*	**Bitmasking**	■		■
56	Check if there exists a subsequence with sum K	*Recursive pick/not-pick with target reduction*	**Recursion Pick/Not-Pick**	■		■
57	Count all subsequences with sum K	*Recursive pick/not-pick counting valid sums*	**Recursion Pick/Not-Pick**	■		■
58	★ **Combination Sum**	*Backtrack, reuse same element (pointer stays), prune when sum>target*	**Backtracking**	■		■

Recursion & Backtracking (17 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
59	★ **Combination Sum II**	*Backtrack, sort array, skip duplicates at same recursion level*	**Backtracking + Dedup**	■		■
60	★ **Subsets I**	*Backtrack include/exclude each index*	**Backtracking**	■		■
61	Subsets II	*Sort array, backtrack skipping duplicate elements at same level*	**Backtracking + Dedup**	■		■
62	Combination Sum III	*Backtrack choosing k numbers 1-9 summing to target, prune early*	**Backtracking**	■		■
63	★ **Letter Combinations of a Phone Number**	*Backtrack building string using digit-to-letters map*	**Backtracking**	■		■
64	★ **Palindrome partitioning**	*Backtrack, try every prefix that's a palindrome, recurse on rest*	**Backtracking**	■		■
65	★ **Word Search**	*DFS/backtrack from each cell, mark visited, restore on backtrack*	**Backtracking / DFS on Grid**	■		■
66	★ **N Queen**	*Backtrack row by row, track attacked cols/diagonals with sets*	**Backtracking**	■		■
67	Rat in a Maze	*DFS/backtrack in 4 directions, mark visited cells*	**Backtracking / DFS on Grid**	■		■
68	M Coloring Problem	*Backtrack assign colors, check adjacency constraint before recursing*	**Backtracking**	■		■
69	★ **Sudoku Solver**	*Backtrack try 1-9 per empty cell, validate row/col/box, undo on fail*	**Backtracking**	■		■

Linked List (44 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
70	Introduction to Singly LinkedList	*Node class with data & next pointer*	**LL Basics**	■		■
71	Traversal in Linked List	*Iterate with a temp pointer until null*	**LL Basics**	■		■
72	Deletion in Linked List	*Relink previous node's next to skip target node*	**LL Basics**	■		■

Linked List (44 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
73	Insertion in Linked List	*Relink pointers to splice new node in*	**LL Basics**	■		■
74	Deletion of the head of LL	*Move head pointer to head.next*	**LL Basics**	■		■
75	Deletion of the tail of Linked List	*Traverse to second-last node, set its next to null*	**LL Basics**	■		■
76	Deletion of the Kth element of Linked List	*Traverse to (k-1)th node, relink around kth*	**LL Basics**	■		■
77	Delete the element with value X	*Traverse tracking prev, relink when node.val==X found*	**LL Basics**	■		■
78	Insertion at the head of Linked List	*New node's next = old head, update head*	**LL Basics**	■		■
79	Insertion at the tail of Linked List	*Traverse to last node, set its next to new node*	**LL Basics**	■		■
80	Insertion at the Kth position of Linked List	*Traverse to (k-1)th node, splice new node after it*	**LL Basics**	■		■
81	Insertion before the value X in Linked List	*Traverse tracking prev, insert new node before node with val X*	**LL Basics**	■		■
82	Deletion in Doubly LL	*Relink both prev and next pointers around deleted node*	**DLL Basics**	■		■
83	Insertion in DLL	*Set both forward and backward links when splicing new node*	**DLL Basics**	■		■
84	Convert Array to Doubly Linked List	*Iterate array, link each node's next/prev to neighbors*	**DLL Basics**	■		■
85	Delete head of Doubly Linked List	*Move head to head.next, set new head.prev = null*	**DLL Basics**	■		■
86	Delete Tail of Doubly Linked List	*Move tail to tail.prev, set new tail.next = null*	**DLL Basics**	■		■
87	Delete Kth Element of Doubly Linked List	*Traverse to kth node, relink prev.next and next.prev*	**DLL Basics**	■		■
88	Removing given node in Doubly Linked List	*Directly relink node.prev.next and node.next.prev*	**DLL Basics**	■		■
89	Insert node before head in Doubly Linked List	*Same as insertion at head, update prev/next both ways*	**DLL Basics**	■		■
90	Insert node before tail in Doubly Linked List	*Traverse to tail.prev, splice new node before tail*	**DLL Basics**	■		■

Linked List (44 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
91	Insert node before (kth node) in Doubly Linked List	*Traverse to kth node, link new node before it both ways*	**DLL Basics**	■		■
92	Insert before given node in Doubly Linked List	*Directly relink using given node's prev pointer*	**DLL Basics**	■		■
93	★ **Add two numbers in Linked List**	*Traverse both lists simultaneously with carry, build result list*	**LL Simulation**	■		■
94	Segregate odd and even nodes in Linked List	*Two dummy lists for odd/even index nodes, merge at end*	**LL Two-Pointer Split**	■		■
95	Sort a Linked List of 0's 1's and 2's	*Count/relink into 3 buckets or single-pass 3 dummy heads*	**LL Bucketing**	■		■
96	★ **Remove Nth node from the back of the LL**	*Two pointers, advance fast n steps ahead, move both till fast ends*	**Fast & Slow Pointers**	■		■
97	★ **Reverse a LL**	*Iteratively reverse next pointers with prev/curr/next trio*	**LL Reversal**	■		■
98	Add one to a number represented by Linked List	*Reverse list, add 1 with carry, reverse back*	**LL Reversal**	■		■
99	★ **Find Middle of Linked List**	*Slow/fast pointer, slow moves 1 fast moves 2 steps*	**Fast & Slow Pointers**	■		■
100	Delete the middle node in LL	*Slow/fast pointer to find middle, delete via prev.next*	**Fast & Slow Pointers**	■		■
101	★ **Check if LL is palindrome or not**	*Find middle, reverse second half, compare both halves*	**Fast & Slow + Reversal**	■		■
102	★ **Find the intersection point of Y LL**	*Two pointers switch heads after reaching end, meet at intersection*	**Two Pointers**	■		■
103	★ **Detect a loop in Linked List**	*Floyd's cycle detection: slow/fast pointers meet if cycle exists*	**Floyd's Cycle Detection**	■		■
104	★ **Find the starting point in LL**	*Floyd's algo: after meeting, move one pointer to head, advance both by 1*	**Floyd's Cycle Detection**	■		■
105	Length of loop in LL	*After Floyd's meeting point, traverse cycle counting steps back to it*	**Floyd's Cycle Detection**	■		■
106	★ **Reverse LL in group of given size K**	*Recursively reverse first k nodes, connect to reversed remainder*	**LL Recursion**	■		■

Linked List (44 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
107	Rotate a Linked List	*Find length, connect tail to head (circular), break at new tail = length-k*	**LL Simulation**	■		■
108	**★ Merge two Sorted Lists**	*Two pointers, compare and link smaller node, splice remainder*	**Two Pointers / Merge**	■		■
109	Flattening of LL	*Merge child lists pairwise/recursively like merge two sorted lists*	**Merge Recursion**	■		■
110	Sort LL	*Merge sort on linked list: find middle, recursively sort halves, merge*	**Merge Sort on LL**	■		■
111	**★ Clone a LL with random and next pointer**	*Interweave cloned nodes, set random via original.next, then separate lists*	**LL + Hashing**	■		■
112	Delete all occurrences of a key in DLL	*Traverse, relink prev/next around every node matching key*	**DLL Basics**	■		■
113	Remove duplicated from sorted DLL	*Traverse, skip node if value equals next, relink pointers*	**DLL Basics**	■		■

Bit Manipulation (8 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
114	Introduction to Bits and Tricks	*AND/OR/XOR/shift basics, check/set/clear bit using masks*	**Bit Basics**	■		■
115	Minimum Bit Flips to Convert Number	*XOR both numbers, count set bits in result*	**XOR / Bit Count**	■		■
116	**★ Single Number - I**	*XOR all elements, pairs cancel out leaving the single number*	**XOR**	■		■
117	Single Number - II	*Bit counting mod 3 across all numbers, or digit DP on bits*	**Bit Counting**	■		■
118	Single Number - III	*XOR all to get xor of two uniques, split by a differing set bit*	**XOR + Bit Split**	■		■
119	Divide two numbers without multiplication and division	*Repeated doubling (bit shifts) to subtract largest multiples of divisor*	**Bit Shifts**	■		■

Bit Manipulation (8 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
120	Power Set	*Bitmask 0 to 2^n-1, include element i if bit i set*	**Bitmasking**	■		■
121	XOR of numbers in a given range	*XOR(1..n) pattern by n%4, then XOR(1..R) ^ XOR(1..L-1)*	**XOR Pattern**	■		■

Greedy (12 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
122	Assign Cookies	*Sort both arrays, greedily assign smallest sufficient cookie*	**Greedy + Sort**	■		■
123	Lemonade Change	*Greedy: prefer giving one $10+$5 over three $5s when possible*	**Greedy Simulation**	■		■
124	★ **Jump Game - I**	*Greedy track farthest reachable index while iterating*	**Greedy**	■		■
125	Shortest Job First	*Sort by burst time, compute waiting/turnaround times greedily*	**Greedy Scheduling**	■		■
126	★ **Job sequencing Problem**	*Sort by profit desc, greedily place each job in latest free slot before deadline*	**Greedy + Sort**	■		■
127	★ **N meetings in one room**	*Sort by end time, greedily pick meetings not overlapping last picked*	**Greedy Interval Scheduling**	■		■
128	★ **Non-overlapping Intervals**	*Sort by end time, greedily keep non-overlapping, count removals*	**Greedy Interval Scheduling**	■		■
129	★ **Insert Interval**	*Add non-overlapping intervals before/after, merge overlapping ones in middle*	**Interval Merge**	■		■
130	★ **Minimum number of platforms required for a railway**	*Sort arrivals & departures separately, two-pointer sweep counting overlaps*	**Greedy Two-Pointer Sweep**	■		■
131	Valid Paranthesis Checker	*Track min/max open count feasible while scanning, handle '*' as wildcard*	**Greedy Range Tracking**	■		■
132	★ **Candy**	*Two passes (left-to-right, right-to-left) enforcing neighbor rating rules*	**Greedy Two-Pass**	■		■

Greedy (12 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
133	Maximum Points You Can Obtain from Cards	*Sliding window: fix window of size (n-k) to exclude from middle*	**Sliding Window**	■		■

Sliding Window & Two Pointers (9 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
134	★ **Longest Substring Without Repeating Characters**	*Sliding window + set/map of last seen index, shrink on duplicate*	**Sliding Window**	■		■
135	★ **Max Consecutive Ones III**	*Sliding window, allow up to k zeros, shrink when zeros exceed k*	**Sliding Window**	■		■
136	Fruit Into Baskets	*Sliding window with map of last 2 fruit types, shrink when 3rd type appears*	**Sliding Window**	■		■
137	Longest Substring With At Most K Distinct Characters	*Sliding window + frequency map, shrink when distinct count > k*	**Sliding Window**	■		■
138	★ **Longest Repeating Character Replacement**	*Sliding window tracking max freq char, shrink when (len-maxFreq)>k*	**Sliding Window**	■		■
139	★ **Minimum Window Substring**	*Sliding window with need/have counts, shrink while all chars satisfied*	**Sliding Window**	■		■
140	Number of Substrings Containing All Three Characters	*Sliding window, for each right track last seen a,b,c, add min index+1*	**Sliding Window**	■		■
141	Binary Subarrays With Sum	*atMost(S)-atMost(S-1) sliding window trick*	**Sliding Window (atMost trick)**	■		■
142	Count number of Nice subarrays	*Same as subarray sum K but on parity (odd count) via atMost technique*	**Sliding Window (atMost trick)**	■		■

Stacks & Queues (22 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
143	Implement Stack using Arrays	*Array + top index, push increments, pop decrements*	**Stack Basics**	■		■

Stacks & Queues (22 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
144	Implement Queue using Arrays	*Circular array with front/rear indices*	**Queue Basics**	■		■
145	Implement Stack using Queue	*Use one queue, rotate elements after each push to reverse order*	**Stack/Queue Conversion**	■		■
146	Implement Queue using Stack	*Two stacks: push to stack1, pop from stack2 (transfer when empty)*	**Stack/Queue Conversion**	■		■
147	Implement stack using Linkedlist	*Push/pop at head of singly linked list, O(1)*	**Stack Basics**	■		■
148	Implement queue using Linkedlist	*Maintain head & tail pointers, enqueue at tail, dequeue at head*	**Queue Basics**	■		■
149	★ **Balanced Paranthesis**	*Stack push opening brackets, pop & match on closing bracket*	**Stack Matching**	■		■
150	★ **Next Greater Element**	*Monotonic decreasing stack from right to left, store indices/values*	**Monotonic Stack**	■		■
151	Next Greater Element - 2	*Circular array: iterate 2n times with monotonic stack*	**Monotonic Stack (Circular)**	■		■
152	★ **Asteroid Collision**	*Stack simulate collisions, pop smaller when right-moving meets left-moving*	**Stack Simulation**	■		■
153	Sum of Subarray Minimums	*Monotonic stack find previous/next smaller element for contribution*	**Monotonic Stack**	■		■
154	Sum of Subarray Ranges	*Sum of subarray maximums minus sum of subarray minimums, both via monotonic stack*	**Monotonic Stack**	■		■
155	Remove K Digits	*Monotonic increasing stack, pop larger digits while k remains*	**Monotonic Stack**	■		■
156	★ **Implement Min Stack**	*Auxiliary stack tracking min, or store encoded value for O(1) space*	**Stack Design**	■		■
157	★ **Sliding Window Maximum**	*Monotonic decreasing deque of indices, pop out-of-window from front*	**Monotonic Deque**	■		■
158	★ **Trapping Rainwater**	*Two pointers with leftMax/rightMax, or precomputed prefix/suffix max arrays*	**Two Pointers**	■		■
159	★ **Largest rectangle in a histogram**	*Monotonic increasing stack, find previous/next smaller element per bar*	**Monotonic Stack**	■		■

Stacks & Queues (22 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
160	Maximum Rectangles	*Per row treat as histogram heights, apply largest rectangle in histogram*	**Monotonic Stack per Row**	■		■
161	Stock span problem	*Monotonic decreasing stack of (price,span), pop while price <= current*	**Monotonic Stack**	■		■
162	Celebrity Problem	*Two-pointer/stack elimination: candidate must have 0 out-degree, n-1 in-degree*	**Two-Pointer Elimination**	■		■
163	★ **LRU Cache**	*HashMap + doubly linked list, move to front on access, evict from back*	**HashMap + DLL Design**	■		■
164	LFU Cache	*HashMap of freq lists (each a DLL) + min-freq pointer for O(1) ops*	**HashMap + DLL Design**	■		■

Binary Trees (30 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
165	Introduction	*Node with left/right child pointers*	**Tree Basics**	■		■
166	★ **Inorder Traversal**	*Left, root, right — recursive or iterative with stack*	**DFS Traversal**	■		■
167	★ **Preorder Traversal**	*Root, left, right — recursive or iterative with stack*	**DFS Traversal**	■		■
168	★ **Postorder Traversal**	*Left, right, root — recursive or two-stack iterative*	**DFS Traversal**	■		■
169	★ **Level Order Traversal**	*BFS using queue, process level by level*	**BFS Traversal**	■		■
170	Pre, Post, Inorder in one traversal	*Single stack storing (node,state 1/2/3) to emit all 3 orders together*	**DFS with State Stack**	■		■
171	★ **Maximum Depth in BT**	*Recursive: 1 + max(depth(left), depth(right))*	**Tree Recursion**	■		■
172	Check if two trees are identical or not	*Recursive compare val, left subtree, right subtree*	**Tree Recursion**	■		■
173	★ **Check for balanced binary tree**	*Recursive height calc, return -1 early if imbalance found*	**Tree Recursion**	■		■
174	★ **Diameter of Binary Tree**	*Recursive height calc, update global max of left height + right height*	**Tree Recursion**	■		■

Binary Trees (30 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
175	★ **Maximum path sum**	*Recursive: track max single-side gain, update global max with left+right+node*	**Tree Recursion**	■		■
176	Check for symmetrical BTs	*Recursive compare mirrored pairs (left.left vs right.right)*	**Tree Recursion**	■		■
177	★ **Zig Zag or Spiral Traversal**	*BFS with level order, reverse alternate levels*	**BFS Level Order**	■		■
178	Boundary Traversal	*Left boundary + leaves (L to R) + right boundary reversed, avoid duplicates*	**DFS + Simulation**	■		■
179	★ **Vertical Order Traversal**	*BFS/DFS with (row,col) map, sort by col then row then value*	**BFS/DFS + Sorting**	■		■
180	★ **Top View of BT**	*BFS with horizontal distance map, keep first node seen per column*	**BFS + HD Map**	■		■
181	Bottom view of BT	*BFS with horizontal distance map, keep last node seen per column*	**BFS + HD Map**	■		■
182	★ **Right/Left View of BT**	*DFS level order, record first node encountered per level (from desired side)*	**DFS Level Order**	■		■
183	Print root to node path in BT	*DFS backtracking, push node to path, pop if target not in subtree*	**DFS Backtracking**	■		■
184	★ **LCA in BT**	*Recursive: if node is p, q, or null return it; combine left/right results*	**Tree Recursion**	■		■
185	Maximum Width of BT	*BFS with index numbering (2*i, 2*i+1), width = last-first index in level*	**BFS + Indexing**	■		■
186	Print all nodes at a distance of K in BT	*Build parent map via BFS/DFS, then BFS from target node K levels*	**Parent Map + BFS**	■		■
187	Minimum time taken to burn the BT from a given Node	*Build parent map, BFS from target counting levels until all visited*	**Parent Map + BFS**	■		■
188	Count total nodes in a complete BT	*Compare left/right height; if equal use 2^h-1 formula, else recurse*	**Height Comparison**	■		■
189	Requirements needed to construct a unique BT	*Need inorder + (preorder or postorder); inorder alone is insufficient*	**Tree Theory**	■		■

Binary Trees (30 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
190	★ **Construct a BT from Preorder and Inorder**	*Recursive with inorder index map, preorder gives root, split inorder*	**Tree Reconstruction**	■		■
191	Construct a BT from Postorder and Inorder	*Recursive with inorder index map, postorder end gives root*	**Tree Reconstruction**	■		■
192	★ **Serialize and De-serialize BT**	*Level order/preorder with null markers, encode as string, decode via queue*	**Tree Encoding**	■		■
193	Morris Inorder Traversal	*Threaded binary tree: temporarily link predecessor's right to current, O(1) space*	**Morris Traversal (O(1) space)**	■		■
194	Morris Preorder Traversal	*Same threading as Morris inorder, print node before creating the thread*	**Morris Traversal (O(1) space)**	■		■

Binary Search Trees (14 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
195	Introduction to BST	*Left subtree < node < right subtree property*	**BST Basics**	■		■
196	Search in BST	*Compare target with node, go left/right, O(h)*	**BST Property**	■		■
197	Floor and Ceil in a BST	*Traverse comparing val, track closest <= (floor) or >= (ceil)*	**BST Property**	■		■
198	Insert a given node in BST	*Traverse to correct null spot per BST property, attach new node*	**BST Property**	■		■
199	★ **Delete a node in BST**	*Find node; if 2 children, replace with inorder successor/predecessor then delete it*	**BST Property**	■		■
200	★ **Kth Smallest and Largest element in BST**	*Inorder traversal (gives sorted order), pick kth element*	**Inorder Traversal**	■		■
201	★ **Check if a tree is a BST or not**	*Recursive with min/max bounds passed down*	**Tree Recursion (Bounds)**	■		■
202	★ **LCA in BST**	*Use BST property: branch left/right based on where p,q lie relative to node*	**BST Property**	■		■

Binary Search Trees (14 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
203	Construct a BST from a preorder traversal	*Recursive with upper bound, or use stack-based O(n) approach*	**BST Reconstruction**	■		■
204	Inorder successor and predecessor in BST	*Traverse using BST property to find closest greater/smaller node*	**BST Property**	■		■
205	★ **BST iterator**	*Controlled inorder traversal using explicit stack, push left spine*	**Controlled Inorder (Stack)**	■		■
206	Two sum in BST	*Inorder traversal to array then two-pointer, or BST iterator forward+backward*	**BST Iterator (Two Pointer)**	■		■
207	Correct BST with two nodes swapped	*Inorder traversal, find two nodes violating sorted order, swap values*	**Inorder Traversal**	■		■
208	Largest BST in Binary Tree	*Postorder DP returning (isBST, min, max, size) per subtree*	**Postorder DP**	■		■

Heaps (10 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
209	Heaps (Theory Video)	*Complete binary tree, array-based, parent i -> children 2i+1,2i+2*	**Heap Basics**	■		■
210	Heapify Algorithm	*Sift down: compare node with children, swap with larger/smaller, repeat*	**Heap Basics**	■		■
211	Build heap from a given Array	*Heapify from last non-leaf node up to root, O(n)*	**Heap Basics**	■		■
212	Implement Min Heap	*Array based, insert bubbles up, extractMin sifts down after swap with last*	**Heap Basics**	■		■
213	Implement Max Heap	*Array based, insert bubbles up, extractMax sifts down after swap with last*	**Heap Basics**	■		■
214	Check if an array represents a min heap	*Verify parent[i] <= children for every non-leaf index*	**Heap Property Check**	■		■
215	Convert Min Heap to Max Heap	*Re-heapify array with max-heap comparator from last non-leaf up*	**Heapify**	■		■

Heaps (10 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
216	★ **Heap Sort**	*Build max heap, repeatedly swap root with last, sift down, shrink heap*	**Heap Sort**	■		■
217	★ **K-th Largest element in an array**	*Min-heap of size k, or quickselect for O(n) average*	**Min-Heap / Quickselect**	■		■
218	★ **Kth largest element in a stream of running integers**	*Maintain min-heap of size k, top is answer after each insert*	**Min-Heap**	■		■

Graphs (44 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
219	Introduction to Graph	*Adjacency list/matrix representation, directed/undirected, weighted*	**Graph Basics**	■		■
220	★ **Traversal Techniques**	*BFS (queue) and DFS (recursion/stack) fundamentals*	**BFS / DFS**	■		■
221	Connected Components	*Run BFS/DFS from each unvisited node, count runs*	**BFS / DFS**	■		■
222	★ **Number of provinces**	*DFS/BFS on adjacency matrix, count connected components*	**BFS / DFS**	■		■
223	★ **Number of islands**	*DFS/BFS from each unvisited land cell in 4/8 directions, count runs*	**BFS / DFS on Grid**	■		■
224	Flood fill algorithm	*DFS/BFS from start cell, recolor matching-color neighbors*	**BFS / DFS on Grid**	■		■
225	Number of enclaves	*DFS from boundary land cells to mark escapable, count remaining land*	**DFS from Boundary**	■		■
226	★ **Rotten Oranges**	*Multi-source BFS from all rotten oranges simultaneously, track time/levels*	**Multi-source BFS**	■		■
227	Distance of nearest cell having one	*Multi-source BFS from all 1-cells, track distance level by level*	**Multi-source BFS**	■		■
228	Surrounded Regions	*DFS from boundary O's to mark safe, flip remaining O's to X*	**DFS from Boundary**	■		■
229	Number of distinct islands	*DFS storing normalized shape (relative coords) per island, count unique in set*	**DFS + Shape Hashing**	■		■

Graphs (44 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
230	★ **Detect a cycle in an undirected graph**	*BFS/DFS tracking parent, cycle if visited neighbor isn't parent*	**BFS/DFS Cycle Detection**	■		■
231	★ **Bipartite graph**	*BFS/DFS 2-coloring, conflict if adjacent nodes same color*	**BFS/DFS 2-Coloring**	■		■
232	★ **Topological sort or Kahn's algorithm**	*BFS on in-degree 0 nodes (Kahn's) or DFS finish-time stack*	**Topological Sort (Kahn's)**	■		■
233	★ **Detect a cycle in a directed graph**	*DFS with visited + pathVisited arrays, cycle if pathVisited neighbor hit*	**DFS Cycle Detection**	■		■
234	Find eventual safe states	*DFS detect cycle nodes (unsafe), reverse-check via pathVisited marking*	**DFS Cycle Detection**	■		■
235	★ **Course Schedule I**	*Topological sort (Kahn's); if all nodes processed, no cycle -> possible*	**Topological Sort**	■		■
236	★ **Course Schedule II**	*Kahn's algorithm, collect topological order as answer*	**Topological Sort**	■		■
237	★ **Alien Dictionary**	*Build graph from adjacent word char comparisons, topological sort gives order*	**Topological Sort**	■		■
238	Shortest path in DAG	*Topological sort order, relax edges in that order, O(V+E)*	**Topological Sort + Relax**	■		■
239	Shortest path in undirected graph with unit weights	*BFS from source, track distance array*	**BFS Shortest Path**	■		■
240	★ **Word ladder I**	*BFS over word transformations (change 1 char at a time), track level as steps*	**BFS**	■		■
241	Word ladder II	*BFS to find shortest length, then DFS/backtrack to reconstruct all shortest paths*	**BFS + Backtracking**	■		■
242	★ **Dijkstra's algorithm**	*Min-heap/priority queue, relax edges greedily by shortest known distance*	**Dijkstra (Priority Queue)**	■		■
243	Print Shortest Path	*Dijkstra with parent array, backtrack from destination to source*	**Dijkstra + Parent Array**	■		■
244	Shortest Distance in a Binary Maze	*BFS (unit weights) from source, track distance grid*	**BFS on Grid**	■		■
245	★ **Path with minimum effort**	*Binary search on effort + BFS feasibility, or Dijkstra minimizing max edge diff*	**Binary Search + BFS / Dijkstra**	■		■

Graphs (44 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
246	★ **Cheapest flight within K stops**	*Modified Bellman-Ford limited to K+1 relaxation rounds*	**Bellman-Ford (Bounded)**	■		■
247	Minimum multiplications to reach end	*BFS over value space (0-99999), each state multiply by given array elements*	**BFS on State Space**	■		■
248	Number of ways to arrive at destination	*Dijkstra + count ways array, add counts when equal shortest distance found*	**Dijkstra + Counting**	■		■
249	★ **Bellman ford algorithm**	*Relax all edges V-1 times, check for negative cycle on Vth pass*	**Bellman-Ford**	■		■
250	★ **Floyd warshall algorithm**	*All-pairs shortest path, DP via intermediate node k in triple loop*	**Floyd-Warshall**	■		■
251	Find the city with the smallest number of neighbors	*Floyd Warshall all-pairs distances, count reachable cities within threshold*	**Floyd-Warshall**	■		■
252	MST theory	*Minimum spanning tree connects all nodes with min total edge weight, no cycles*	**MST Theory**	■		■
253	★ **Disjoint Set**	*Union-Find with path compression + union by rank/size, near O(1) ops*	**Union-Find (DSU)**	■		■
254	★ **Find the MST weight**	*Prim's (min-heap greedy) or Kruskal's (sort edges + DSU)*	**Prim's / Kruskal's**	■		■
255	Number of operations to make network connected	*Count components via DSU; answer = components-1 if enough extra edges*	**DSU**	■		■
256	★ **Accounts merge**	*DSU merge accounts sharing an email, group emails by root parent*	**DSU**	■		■
257	Number of islands II	*DSU, union adjacent land cells online, track component count after each add*	**DSU (Online)**	■		■
258	Making a large island	*DSU union existing islands, then for each 0-cell check union of neighboring island sizes*	**DSU**	■		■
259	Most stones removed with same row or column	*DSU union stones sharing row/col, answer = stones - number of components*	**DSU**	■		■
260	★ **Kosaraju's algorithm**	*DFS finish-order stack, transpose graph, DFS in stack order for SCCs*	**SCC (Kosaraju's)**	■		■

Graphs (44 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
261	★ **Bridges in graph**	*DFS with discovery/low-link times, edge is bridge if low[child] > disc[node]*	**Tarjan's Bridge-Finding**	■		■
262	★ **Articulation point in graph**	*DFS with discovery/low-link times, check low[child] >= disc[node] condition*	**Tarjan's Low-Link**	■		■

Dynamic Programming (47 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
263	Introduction to DP	*Overlapping subproblems + optimal substructure, memoization or tabulation*	**DP Theory**	■		■
264	★ **Climbing stairs**	*dp[i] = dp[i-1] + dp[i-2], Fibonacci pattern*	**1D DP**	■		■
265	★ **Frog Jump**	*dp[i] = min(dp[i-1]+cost, dp[i-2]+cost) choosing 1 or 2 step jump*	**1D DP**	■		■
266	Frog jump with K distances	*dp[i] = min over j=1..K of dp[i-j] + |h[i]-h[i-j]|*	**1D DP**	■		■
267	★ **Maximum sum of non adjacent elements**	*dp[i] = max(dp[i-1], dp[i-2]+a[i]), house robber pattern*	**1D DP**	■		■
268	★ **House robber**	*Same as non-adjacent sum DP applied circularly for two passes*	**1D DP (Circular)**	■		■
269	Ninja's training	*DP over days x activities, exclude same activity as previous day*	**DP on Grid/State**	■		■
270	★ **Grid unique paths**	*dp[i][j] = dp[i-1][j] + dp[i][j-1], or combinatorics nCr*	**DP on Grid**	■		■
271	Unique paths II	*Same DP as grid unique paths, set dp=0 at obstacle cells*	**DP on Grid**	■		■
272	Minimum Falling Path Sum	*dp[i][j] = a[i][j] + min(dp[i-1][j-1],dp[i-1][j],dp[i-1][j+1])*	**DP on Grid**	■		■
273	Triangle	*Bottom-up DP: dp[i][j] = a[i][j] + min(dp[i+1][j], dp[i+1][j+1])*	**DP on Grid**	■		■
274	Cherry pickup II	*3D DP over row and two robot columns, consider all 9 move combos*	**DP on Grid (3D)**	■		■

Dynamic Programming (47 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
275	★ **Best time to buy and sell stock**	*Track min price so far, update max profit = price - minPrice*	**DP on Stocks**	■		■
276	★ **Best time to buy and sell stock II**	*Greedy sum all positive consecutive differences (or DP with buy/sell states)*	**Greedy / DP on Stocks**	■		■
277	Best time to buy and sell stock III	*DP with at-most-2-transaction states (buy1,sell1,buy2,sell2)*	**DP on Stocks**	■		■
278	Best time to buy and sell stock IV	*DP with 2D state (day, transaction count) tracking buy/sell*	**DP on Stocks**	■		■
279	Best time to buy and sell stock with transaction fees	*DP tracking cash/hold states, subtract fee on sell*	**DP on Stocks**	■		■
280	★ **Subset sum equals to target**	*dp[i][sum] = pick or not-pick boolean DP*	**DP on Subsequences (0/1 Knapsack)**	■		■
281	★ **Partition equal subset sum**	*Subset sum DP for target = totalSum/2*	**DP on Subsequences (0/1 Knapsack)**	■		■
282	Partition a set into two subsets with minimum absolute sum difference	*Subset sum DP up to totalSum/2, find closest achievable sum*	**DP on Subsequences**	■		■
283	Count subsets with sum K	*dp[i][sum] = dp[i-1][sum] + dp[i-1][sum-a[i]], handle zeros carefully*	**DP on Subsequences**	■		■
284	Count partitions with given difference	*Convert to count subsets with sum = (total+diff)/2*	**DP on Subsequences**	■		■
285	★ **0 and 1 Knapsack**	*dp[i][w] = max(skip, value[i]+dp[i-1][w-weight[i]])*	**0/1 Knapsack**	■		■
286	★ **Minimum coins**	*Unbounded knapsack: dp[amt] = min(dp[amt-coin]+1) over all coins*	**Unbounded Knapsack**	■		■
287	Target sum	*Convert to count subsets with sum = (total+target)/2*	**DP on Subsequences**	■		■
288	★ **Coin change II**	*Unbounded knapsack counting combinations: dp[i][amt] += dp[i-1][amt]*	**Unbounded Knapsack**	■		■
289	Unbounded knapsack	*dp[i][w] = max(skip, value[i]+dp[i][w-weight[i]]) — same row reuse*	**Unbounded Knapsack**	■		■
290	Rod cutting problem	*Unbounded knapsack where weight=length, value=price per piece*	**Unbounded Knapsack**	■		■

Dynamic Programming (47 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
291	★ **Longest Increasing Subsequence**	*dp[i] = 1+max(dp[j]) for j<i with a[j]<a[i]; or patience sort O(n log n)*	**DP on LIS**	■		■
292	Print Longest Increasing Subsequence	*LIS DP with parent/hash array to reconstruct the actual sequence*	**DP on LIS**	■		■
293	Largest Divisible Subset	*Sort array, LIS-style DP where a[i]%a[j]==0 replaces a[j]<a[i]*	**DP on LIS**	■		■
294	Longest String Chain	*Sort by length, DP checking predecessor by removing one char*	**DP on LIS**	■		■
295	Longest Bitonic Subsequence	*Compute LIS from left and LIS from right (as LDS), combine at each index*	**DP on LIS**	■		■
296	Number of Longest Increasing Subsequences	*LIS DP with count array, sum counts of all max-length-achieving predecessors*	**DP on LIS**	■		■
297	★ **Longest common subsequence**	*2D DP: match chars diagonally +1, else max(skip one from either string)*	**DP on Strings**	■		■
298	Longest common substring	*2D DP: match resets on mismatch to 0, track global max*	**DP on Strings**	■		■
299	★ **Longest palindromic subsequence**	*LCS of string with its reverse*	**DP on Strings (LCS variant)**	■		■
300	Minimum insertions to make string palindrome	*n - LPS(string), insertions fill the non-palindromic gap*	**DP on Strings**	■		■
301	Minimum insertions or deletions to convert string A to B	*(lenA - LCS) deletions + (lenB - LCS) insertions*	**DP on Strings**	■		■
302	Shortest common supersequence	*lenA + lenB - LCS length, reconstruct via LCS backtracking*	**DP on Strings**	■		■
303	Distinct subsequences	*2D DP: dp[i][j] = dp[i-1][j-1](if match)+dp[i-1][j]*	**DP on Strings**	■		■
304	★ **Edit distance**	*2D DP: match diagonal, else 1+min(insert,delete,replace)*	**DP on Strings**	■		■
305	★ **Wildcard matching**	*2D DP handling '?' as any char, '*' as dp[i-1][j] or dp[i][j-1]*	**DP on Strings**	■		■

Dynamic Programming (47 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
306	★ **Matrix chain multiplication**	*Interval DP over (i,j), try every split point k, minimize cost*	**Interval DP (MCM)**	■		■
307	Minimum cost to cut the stick	*Interval DP with added boundary cuts, min cost = length + sub-costs*	**Interval DP (MCM)**	■		■
308	★ **Burst balloons**	*Interval DP, treat k as last balloon burst in range (i,j)*	**Interval DP (MCM)**	■		■
309	Palindrome partitioning II	*DP: minCuts[i] = min cuts for prefix, check palindrome via precomputed table*	**DP + Palindrome Precompute**	■		■

Tries (6 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
310	★ **Trie Implementation and Operations**	*Node array/map of 26 children + isEnd flag, insert/search char by char*	**Trie Basics**	■		■
311	Trie Implementation and Advanced Operations	*Add countEndsWith/countPrefix fields per node for prefix queries*	**Trie + Counts**	■		■
312	Longest Word with All Prefixes	*Trie + DFS/BFS checking every prefix node isEnd=true along the way*	**Trie + DFS**	■		■
313	Number of distinct substrings in a string	*Insert every suffix into Trie, count new nodes created*	**Trie of Suffixes**	■		■
314	★ **Maximum XOR of two numbers in an array**	*Binary Trie of bits (MSB first), greedily pick opposite bit per number*	**Binary Trie**	■		■
315	Maximum Xor with an element from an array	*Binary Trie, greedily maximize XOR while respecting value <= limit constraint*	**Binary Trie (Offline)**	■		■

String Algorithms (8 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
316	Reverse every word in a string	*Split by spaces, reverse each token or reverse whole then re-reverse words*	**String Simulation**	■		■

String Algorithms (8 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
317	Minimum number of bracket reversals to make an expression balanced	*Count unmatched '(' and ')' after removing matched pairs, answer=ceil(open/2)+ceil(close/2)*	**Stack / Counting**	■		■
318	Count and say	*Iteratively build next term via run-length encoding of previous term*	**String Simulation**	■		■
319	Rabin Karp Algorithm	*Rolling hash of pattern vs text window, verify on hash match*	**Rolling Hash**	■		■
320	Z function	*Z-array: length of longest match with prefix at each index, linear construction*	**Z-Function**	■		■
321	★ **KMP Algorithm or LPS array**	*Build LPS (failure) array, use it to skip re-comparisons on mismatch*	**KMP / LPS Array**	■		■
322	Shortest Palindrome	*KMP LPS on (s + '#' + reverse(s)) to find longest palindromic prefix*	**KMP on Modified String**	■		■
323	Longest happy prefix	*KMP LPS array; last value = length of longest prefix that's also a suffix*	**KMP / LPS Array**	■		■

Maths (3 problems)

#	Problem Name	Short Hint (Optimal Approach)	Pattern / Topic Tag	Completed	Rev. Count	Revisit
324	★ **Print all primes till N**	*Sieve of Eratosthenes: mark multiples of each prime starting from p*p*	**Sieve of Eratosthenes**	■		■
325	Prime factorisation of a Number	*Trial division up to sqrt(n), divide out each prime factor repeatedly*	**Trial Division**	■		■
326	Count primes in range L to R	*Sieve of Eratosthenes up to sqrt(R), then segmented sieve for [L,R]*	**Segmented Sieve**	■		■